

(LEC 34)

Map, Filter & Reduce in JavaScript Assignment

PW EARNERS : Coding and Web Development

By : Nishant Saini Sir

Topics Covered

- map()
- Immutability
- map() vs forEach()
- filter() with conditions
- reduce()
- Accumulator Pattern

Section 1 – map() and Immutability

1. Convert Product Names to Uppercase

Create an array of product names and use map() to create a new array where every product name is converted to uppercase.

Example:

Input:

`["laptop", "mobile", "headphones"]`

Output:

`["LAPTOP", "MOBILE", "HEADPHONES"]`

2. Add a Currency Symbol to Prices

Create an array of product prices and use map() to create a new array where each price is displayed with a ₹ symbol.

Example:

Input:

`[100, 250, 500]`

Output:

`["₹100", "₹250", "₹500"]`

3. Extract User Names

Create an array of user objects containing name and email. Use map() to create a new array containing only the names.

Example:

Input:

```
[  
  { name: "Rahul", email: "rahul@example.com" },  
  { name: "Priya", email: "priya@example.com" }  
]
```

Output:

```
["Rahul", "Priya"]
```

4. Create Updated Product Prices

Create an array of product prices. Use `map()` to create a new array where every price is increased by 10%. Keep the original array unchanged.

Example:

Input:

```
[100, 200, 300]
```

Output:

Original: [100, 200, 300]

New: [110, 220, 330]

5. Update Object Data Immutably

Create an array of user objects with name and role. Use `map()` and the spread operator to create a new array where the role of every user is changed to "developer" without modifying the original array.

Example:

Input:

```
[  
  { name: "Rahul", role: "student" },  
  { name: "Priya", role: "student" }  
]
```

Output:

```
[  
  { name: "Rahul", role: "developer" },  
  { name: "Priya", role: "developer" }  
]
```

6. Add a New Property Using map()

Create an array of product objects containing name and price. Use map() to create a new array where each product also has an inStock property with the value true.

Example:

Input:

```
[
  { name: "Laptop", price: 50000 },
  { name: "Mouse", price: 500 }
]
```

Output:

```
[
  { name: "Laptop", price: 50000, inStock: true },
  { name: "Mouse", price: 500, inStock: true }
]
```

Section 2 – map() vs forEach()

7. Display Technologies Using forEach()

Create an array of frontend technologies and use forEach() to display every technology.

Example:

Input:

```
["HTML", "CSS", "JavaScript"]
```

Output:

HTML

CSS

JavaScript

8. Create a New Array Using map()

Using the same array of frontend technologies, use map() to create a new array where every technology is converted to uppercase.

Example:

Input:

```
["html", "css", "javascript"]
```

Output:

```
["HTML", "CSS", "JAVASCRIPT"]
```

9. Format User Names Using map()

Create an array of names and use map() to add the text "User: " before every name. Display the new array.

Example:

Input:

```
["Rahul", "Priya", "Aman"]
```

Output:

```
["User: Rahul", "User: Priya", "User: Aman"]
```

Section 3 – filter()

10. Filter Available Products

Create an array of product objects containing name and inStock. Use filter() to create a new array containing only the products that are in stock.

Example:

Input:

```
[  
  { name: "Laptop", inStock: true },  
  { name: "Mouse", inStock: false }  
]
```

Output:

```
[  
  { name: "Laptop", inStock: true }  
]
```

11. Filter Users by Role

Create an array of user objects containing name and role. Use filter() to get all users whose role is "developer".

Example:

Input:

```
[  
  { name: "Rahul", role: "developer" },  
  { name: "Priya", role: "student" }  
]
```

Output:

```
[
  { name: "Rahul", role: "developer" }
]
```

12. Filter Expensive Products

Create an array of product objects containing name and price. Use filter() to get products with a price greater than 1000.

Example:

Input:

```
[
  { name: "Mouse", price: 500 },
  { name: "Keyboard", price: 1500 }
]
```

Output:

```
[
  { name: "Keyboard", price: 1500 }
]
```

13. Filter Active Users

Create an array of users containing name and isActive. Use filter() to get only the active users.

Example:

Input:

```
[
  { name: "Rahul", isActive: true },
  { name: "Priya", isActive: false }
]
```

Output:

```
[
  { name: "Rahul", isActive: true }
]
```

14. Filter Gmail Addresses

Create an array of email addresses and use filter() to get only the emails that include "@gmail.com".

Example:

Input:

```
["rahul@gmail.com", "priya@yahoo.com", "aman@gmail.com"]
```

Output:

```
["rahul@gmail.com", "aman@gmail.com"]
```

Section 4 – reduce() and Accumulator Pattern

15. Calculate the Total Cart Price

Create an array of product prices and use reduce() to calculate the total price of all items in the cart.

Example:

Input:

```
[500, 1200, 300]
```

Output:

```
2000
```

16. Count Total Products

Create an array of product names and use reduce() with an accumulator to count the total number of products.

Example:

Input:

```
["Laptop", "Mouse", "Keyboard"]
```

Output:

```
3
```

17. Calculate the Total Quantity

Create an array of cart item objects containing name and quantity. Use reduce() to calculate the total quantity of all items.

Example:

Input:

```
[  
  { name: "Laptop", quantity: 1 },  
  { name: "Mouse", quantity: 2 }  
]
```

Output:

3

18. Calculate Total Order Amount

Create an array of order objects containing amount. Use `reduce()` to calculate the total order amount.

Example:

Input:

```
[  
  { amount: 500 },  
  { amount: 1000 },  
  { amount: 750 }  
]
```

Output:

2250

19. Create a Comma-Separated String

Create an array of frontend technologies and use `reduce()` to combine them into a single comma-separated string.

Example:

Input:

```
["HTML", "CSS", "JavaScript"]
```

Output:

"HTML, CSS, JavaScript"

20. Calculate Final Cart Total

Create an array of cart items containing name, price, and quantity. Use `reduce()` to calculate the final cart total by multiplying the price and quantity of each item.

Example

Input:

```
[  
  { name: "Mouse", price: 500, quantity: 2 },  
  { name: "Keyboard", price: 1000, quantity: 1 }  
]
```

Output:

2000